

Senior QA Engineer Technical Assessment

Electro Pi POS & Inventory Platform

Abanoub Gallah

1 September 2026

Code: <https://github.com/AbanoubGallah/ElectroPi>

I built the framework instead of just describing it. The code is at github.com/AbanoubGallah/ElectroPi and runs on a clean clone with `npm ci && npx playwright test`. It includes a small mock POS app, so there is no environment to set up first. 17 tests pass (8 API, 9 UI) and the Postman collection runs under Newman with 38 assertions.

The brief left a few things open, mainly the validation status codes and the database schema. I have listed what I assumed at the end.

Part 1: UI test automation and architecture

1. Framework choice

I would use Playwright with TypeScript.

The main reason is that it covers the UI and the API in one tool. Most of the instability in UI suites comes from setup, not from the UI itself, and being able to log in and create inventory over the REST API inside the same test file removes a lot of it. The same request context then covers the Part 2 API tests, so we get one framework, one CI setup and one report instead of two of each.

The second reason is multi-tenancy. A browser context in Playwright is a cheap, isolated browser profile, so one test can hold two signed-in merchants at the same time and check that neither can see the other's data. On a platform like this that is the failure I would least like to ship.

Third, debugging. With trace: 'on-first-retry' a failed CI run produces a trace with DOM snapshots, network calls and console output, which you replay locally with `npx playwright show-trace`. In practice this saves more time than any difference in syntax between the three tools.

	Playwright	Cypress	Selenium
Waiting for elements	Automatic	Automatic	Manual <code>WebDriverWait</code>
API testing in the same tool	Yes	<code>cy.request</code>	No
WebKit / Safari (iPad terminals)	Bundled	Experimental	Needs macOS + <code>SafariDriver</code>
Parallel runs and sharding	Included	Paid feature	Needs a Grid
Debugging a CI failure	Trace viewer	Video, time travel	Screenshot and stack trace

I would still pick Selenium if we needed a large real-device farm or if the team wrote tests in several languages, and Cypress if the team already used it and Safari coverage did not matter. Neither applies here.

2. Framework structure

Page Object Model, with component objects for shared UI and Playwright fixtures for setup and teardown.

```
src/
  config/env.ts      environment, credentials, timeouts
  api/               AuthApi, InventoryApi and their response types
  pages/             BasePage, LoginPage, InventoryPage
    components/      ToastComponent, SideNavComponent
  fixtures/          test fixtures: setup, auth, cleanup
  data/              test data factories
tests/
  api/               API suite
  ui/                UI suite
```

`config/env.ts` is the only file that reads `process.env`. It resolves URLs, users and timeouts, and throws a clear error if something is missing. The same suite runs against local, staging or pre-prod by changing environment variables.

`api/` holds thin wrappers over the endpoints. They return the raw response rather than throwing on a non-2xx, otherwise negative tests could not use them. They are also how UI tests set up their data.

In `pages/` I keep to three habits. Locators are created in the constructor and never resolved until they are used, so a React re-render cannot invalidate them. Assertions about behaviour stay in the tests, so the same page object works for a happy path and a validation case without growing if-statements; the only exception is `waitUntilReady()`, which is a wait rather than a check. Anything in the app shell, such as the toast host and the sidebar, becomes a component object so it exists in one place.

`fixtures/` is the part a plain POM usually lacks. A test asks for what it needs:

```
test('...', async ({ inventoryAsAdmin, adminToken, cleanup }) => { ... });
```

The `cleanup` fixture deletes anything the test created even if the test failed, because fixtures always unwind. `adminToken` is worker-scoped, so the suite logs in once per worker instead of once per test. Combinations of setup are declared instead of inherited from a chain of base classes.

`data/` builds payloads: `anItem({ price: -5 })` gives a valid item with one field changed, which keeps negative tests to a line each. SKUs are generated per call, so parallel workers never collide and a re-run does not hit a duplicate.

On the four qualities in the question:

- Scalability comes from the layering plus tags (`@smoke`, `@regression`, `@api`, `@security`) that decide what runs at each pipeline stage. A new module means one page object and one spec.
- Reuse comes from component objects, the API clients and the data factories.
- Maintainability mostly comes from selectors living in one place per page, and from typed responses, so a renamed API field fails the type check instead of an assertion somewhere.
- Debugging comes from traces on retry, `test.step` names that match the test case, and screenshots and video on failure.

I considered the Screenplay pattern. It suits suites with many roles composing many reusable tasks, and I would look at it again if this grew to a dozen roles. Right now it would add a vocabulary the application developers do not use, and Playwright fixtures already give most of the composability people move to Screenplay for.

3. Implementation

From tests/ui/inventory-create-product.spec.ts:

```
test.describe('Inventory - create product', () => {
  test.use({ storageState: { cookies: [], origins: [] } });

  test('Store Admin can create a product and sees a success toast @smoke @regression',
    async ({ loginPage, inventoryPage, cleanup }) => {
      const product = aProductForm();

      await test.step('Log in as a Store Admin', async () => {
        await loginPage.open();
        await loginPage.loginAs(env.users.storeAdmin);
      });

      await test.step("Navigate to the 'Inventory' module", async () => {
        await inventoryPage.nav.goToInventory();
        await inventoryPage.waitUntilReady();
      });

      await test.step("Fill in the 'Product Name' and 'Price'", async () => {
        await inventoryPage.openNewProductForm();
        await inventoryPage.fillProductForm({ name: product.name, price: product.price });
      });

      const response = await test.step("Click 'Save'", async () => {
        return inventoryPage.save();
      });

      await test.step('Assert that a success toast message appears', async () => {
        await inventoryPage.toast.expectSuccess(/saved successfully/i);
      });

      await test.step('Check it really was saved', async () => {
        expect(response.status()).toBe(201);
        const created = (await response.json()) as InventoryItem;
        cleanup.push(created.id);
        expect(created).toMatchObject({
          item_name: product.name,
          price: Number(product.price),
          category_name: 'Electronics',
        });
      });
    });
});
```

```

    });
    await expect(inventoryPage.rowByName(product.name)).toBeVisible();
  });
});
});

```

Each of the five requested steps is a `test.step`, so the HTML report reads like the test case it came from.

I added a sixth step that checks the 201 and the grid row. A success toast only tells you the UI thinks it worked. While building this I changed the API to return 201 without saving anything, and a check that stopped at the toast still passed, while this one failed.

The two page-object methods doing the waiting:

```

// InventoryPage: the page is ready when the spinner has gone and the button
// the data unlocks is enabled. The app enables it only after both
// /inventory/items and /categories have come back.
async waitUntilReady(): Promise<void> {
  await expect(this.loadingSpinner).toBeHidden({ timeout: env.timeouts.action });
  await expect(this.addProductButton).toBeEnabled({ timeout: env.timeouts.action });
}

// The response listener is set up before the click, so a fast response
// cannot be missed.
async save(): Promise<Response> {
  const responsePromise = this.page.waitForResponse(
    (r) => r.url().includes('/api/v1/inventory/items') && r.request().method() === 'POST',
    { timeout: env.timeouts.action },
  );
  await this.saveButton.click();
  return responsePromise;
}

```

The repository also covers a missing price, a negative price, a duplicate SKU, reaching `/inventory` while signed out, and the toast clearing itself.

4. Dynamic elements and flaky tests

A fixed sleep is wrong either way: too short and the test is flaky, too long and the suite is slow. So every wait is a wait for something observable.

Playwright's assertions (`toBeVisible`, `toBeEnabled`, `toHaveText`) poll until they pass or time out, and its actions wait for an element to be visible, stable and enabled before acting. That covers most dynamic rendering. What it does not decide is what "ready" means for a given page, so I define that per page:

- Spinners: wait for `toBeHidden()`, not for a duration.
- Data arriving: wait for what the data enables. On the inventory page the "Add product" button is enabled only after both requests return, so waiting on that one condition covers both.

- Fields filled asynchronously: the category select stays disabled until its options load, so `toBeEnabled()` is the gate before selecting.
- A specific request: `waitForResponse`, set up before the click.

I avoid `networkidle`. A POS dashboard polls, so "no requests for 500ms" may never be true.

For selectors I use `data-testid` first, then role or label, then CSS structure, and never XPath or generated class names like `.css-1x3fh9`. Adding a `testid` to new components is cheap for developers and it is the single change that removes most UI flakiness, so I would ask for it as part of a component's definition of done. Rows are found by SKU or name rather than by index, since with parallel workers "the first row" is not stable.

Isolation matters as much as waiting, and gets misread as a timing problem more often than anything else. Each test gets a fresh browser context, creates its own data with a unique SKU, and cleans up in a fixture instead of at the end of the test body, so cleanup still runs after a failure. No test depends on another running first.

Where the app is not deterministic I make it so: `page.route` to stub third-party calls or force a 500 from the inventory service, and `page.clock` for anything time-dependent such as an end-of-day cutoff. CI pins the browser version through Playwright, and the viewport, timezone and locale, so a currency or date assertion cannot fail because of where a runner is.

Retries are on in CI only, set to 1. Locally there are none, so flakiness shows up while it is cheap to fix. A test that passes on retry is reported as flaky, and that report is the list I work from. A nightly job runs the suite with `--repeat-each=5 --retries=0` to find the failure that happens once in twenty. If a test stays flaky it gets tagged `@quarantine`, taken out of the blocking gate, and given a ticket with an owner and a date.

Some cases I have hit before and what they usually turn out to be:

Symptom	Usual cause	Fix
Passes locally, "element not found" in CI	Assertion ran before the data rendered	Wait on what the data enables
Click hits the wrong element	Layout moved as data arrived	Playwright's actionability checks
Toast assertion fails now and then	Toast auto-dismissed between the wait and reading the text	One assertion that waits and checks the text together
Passes alone, fails in the suite	Shared data or leftover session	Unique data per test, fresh context, cleanup in a fixture
Fails only in one shard	Hidden dependency on another test	Remove shared state

To check there was nothing timing-dependent left, I raised the mock app's artificial latency from 350ms to 2000ms and ran the smoke tests again without changing any code. They passed.

Part 2: API testing strategy

`POST /api/v1/inventory/items`, with `Authorization: Bearer <token>` and the payload from the brief.

1. Test scenarios

These are implemented in `tests/api/inventory-items.spec.ts`. Each one checks the status code, the response body, and then reads the item back, because a test that only checks the status code will pass against an endpoint that returns 201 and saves nothing.

Positive 1: valid payload, Store Admin token. The payload from the brief with a valid token and an existing `category_id: 3`. Expect **201**. The body echoes the fields sent, adds an `id`, `created_at` and `category_name: "Electronics"`, and there is a `Location` header. A following GET on that `id` returns 200 with the same values.

Positive 2: boundary values. `quantity: 0` (valid for a pre-order or an out-of-stock line), `price: 0.01`, and an `item_name` at the 120 character limit. Expect **201** with the values stored as sent: `quantity` is 0 rather than rejected as empty, `price` has not drifted, and the name is not truncated. Boundaries are where validation code is usually thinnest.

Negative 1: missing or invalid token. No `Authorization` header, then a tampered one. Expect **401** for both, nothing created, and no detail about the tenant or schema in the response. This is the first negative test I would write on a write endpoint, since getting it wrong lets anyone write into any merchant's inventory.

Negative 2: invalid field values. One violation per case: `item_name` missing or blank, `sku` missing or with illegal characters, `quantity` of -5 or 2.5, `price` of -25.00 or the string "25.00" or 25.005, and a `category_id` that does not exist. Expect **422** (400 would be fine too, as long as it is documented and consistent) with the field named, for example { "field": "price", "issue": "MUST_BE_GTE_0" }, and nothing saved. The test checks the field and reason, not just the status, because an "expect 422" also passes when the API blames the wrong field. Negative prices and quantities are worth extra attention on a POS, since they affect stock valuation.

Negative 3: duplicate SKU. The same SKU posted twice. Expect **409** naming the SKU, and a follow-up list query returning exactly one record. SKU uniqueness should be per merchant, not global, since two stores may both sell MS-001.

Three more that I would not leave out on a multi-tenant write endpoint, and which are in the repository: a Cashier token gets **403** and not 401, malformed JSON gets **400** and not a 500, and reading another merchant's item gets **404**, since a 403 would confirm the record exists.

Further along I would add rate limiting, oversized payloads, two clients creating the same SKU at once, and a contract test against the OpenAPI spec so a renamed field fails CI.

2. Postman

The collection is in `postman/` and runs under Newman.

Getting a token before the request runs

A pre-request script on the collection, so it runs before every request and no request carries a pasted token. It only fetches a new one when the current one is missing or nearly expired, so a run of 40 requests logs in once.

```
const SKEW_MS = 30 * 1000;
const token = pm.environment.get('access_token');
const expiresAt = Number(pm.environment.get('access_token_expires_at') || 0);
```

```

const stillValid = Boolean(token) && Date.now() < expiresAt - SKEW_MS;

if (stillValid) {
  console.log('[auth] reusing cached token');
} else {
  pm.sendRequest({
    url: pm.environment.get('base_url') + '/api/v1/auth/login',
    method: 'POST',
    header: { 'Content-Type': 'application/json' },
    body: {
      mode: 'raw',
      raw: JSON.stringify({
        email: pm.environment.get('store_admin_email'),
        password: pm.environment.get('store_admin_password')
      })
    }
  }, function (err, res) {
    // Fail here rather than letting every later request return 401.
    if (err) { throw new Error('Token request failed: ' + err); }
    if (res.code !== 200) { throw new Error('Login failed: ' + res.code + ' ' + res.text()); }

    const body = res.json();
    pm.environment.set('access_token', body.access_token);
    pm.environment.set('access_token_expires_at', Date.now() + body.expires_in * 1000);
    pm.environment.set('tenant_id', body.user.tenant_id);
  });
}

```

Around that: bearer auth is set once on the collection to `{{access_token}}` and inherited, with the 401 test overriding it to No Auth for that request only. Credentials are environment variables, the password as a secret type so it is masked and left out of exports, and injected from the secret store in CI. `base_url` is a variable too, so the same collection runs against local or staging. A short pre-request script generates a unique SKU per run so the 409 case is not hit by accident. The requests are ordered create, read back, duplicate, negatives, delete, so a run cleans up after itself.

Tests tab for the 201 response

```

const expectedSku = pm.collectionVariables.get('request_sku');

pm.test('Status code is 201 Created', function () {
  pm.response.to.have.status(201);
});

// Parsed after the status check, so a 500 does not show up as a JSON error.
const body = pm.response.json();

pm.test('Response body returns the same SKU that was sent', function () {

```

```

    pm.expect(body).to.have.property('sku');
    pm.expect(body.sku).to.eql(expectedSku);
  });

  pm.test('Response echoes the submitted item', function () {
    pm.expect(body.item_name).to.eql('Wireless Mouse');
    pm.expect(body.quantity).to.eql(50);
    pm.expect(body.price).to.eql(25);
    pm.expect(body.category_id).to.eql(3);
  });

  pm.test('Server assigned an id and a timestamp', function () {
    pm.expect(body.id).to.be.a('string').and.to.have.lengthOf(36);
    pm.expect(Date.parse(body.created_at)).to.not.be.NaN;
  });

  pm.test('Item belongs to the authenticated tenant', function () {
    pm.expect(body.tenant_id).to.eql(pm.environment.get('tenant_id'));
  });

  pm.test('Location header points at the new resource', function () {
    pm.expect(pm.response.headers.get('Location'))
      .to.include('/api/v1/inventory/items/' + body.id);
  });

  pm.test('Response matches the schema', function () {
    pm.response.to.have.jsonSchema({
      type: 'object',
      required: ['id', 'item_name', 'sku', 'quantity', 'price',
        'category_id', 'tenant_id', 'created_at'],
      properties: {
        id: { type: 'string' },
        item_name: { type: 'string' },
        sku: { type: 'string' },
        quantity: { type: 'integer', minimum: 0 },
        price: { type: 'number', minimum: 0 },
        category_id: { type: 'integer' },
        category_name: { type: ['string', 'null'] },
        tenant_id: { type: 'string' },
        created_at: { type: 'string' }
      }
    });
  });

  pm.collectionVariables.set('created_item_id', body.id);

```

The SKU is compared against the variable that was sent, not a hardcoded "MS-001", which would still pass if the API returned a default or a stale value. The schema check is there to catch a renamed or dropped field, which the value-by-value assertions would miss.

Part 3: CI/CD and database validation

1. Database testing

Full script: `sql/01_verify_item_in_electronics.sql`. The brief names products and categories, and the payload gives the column names, so the query uses only those. Columns that often exist on a platform like this are listed after it as things to confirm, not as assumptions.

```
SELECT
  p.id,
  p.item_name,
  p.sku,
  p.quantity,
  p.price,
  p.category_id,
  c.name AS category_name
FROM products p
INNER JOIN categories c
  ON c.id = p.category_id
WHERE p.sku = :sku
  AND c.name = 'Electronics';
```

An inner join, because if the category reference does not resolve the row should not come back. The filter is on `c.name` and not `p.category_id = 3`: the API was given the id 3, so checking the id only proves the value we sent came back. Joining and checking the name proves it resolves to Electronics, which is the question being asked. `:sku` is bound, not concatenated.

In the test I run a second version of this. The query above returns no rows for three different reasons: never created, created against the wrong category, or a `category_id` that resolves to nothing. "No rows" is not a useful failure message, so the test version returns one row of booleans and names the one that broke:

```
WITH expected AS (
  SELECT CAST(:sku AS TEXT) AS sku, 'Electronics' AS category_name,
    'Wireless Mouse' AS item_name, 50 AS quantity,
    CAST(25.00 AS NUMERIC(12,2)) AS price, 3 AS category_id
),
actual AS (
  -- Left join, so a product whose category_id points at nothing still comes
  -- back with category_name NULL.
  SELECT p.id, p.item_name, p.sku, p.quantity, p.price, p.category_id,
    c.name AS category_name
  FROM products p
  LEFT JOIN categories c ON c.id = p.category_id
```

```

WHERE p.sku = (SELECT sku FROM expected)
)
SELECT
  (SELECT COUNT(*) FROM actual) = 1 AS exactly_one_row,
  a.id IS NOT NULL AS product_exists,
  a.category_name IS NOT NULL AS category_reference_resolves,
  a.category_name = e.category_name AS assigned_to_electronics,
  a.item_name = e.item_name AS item_name_matches,
  a.quantity = e.quantity AS quantity_matches,
  a.price = e.price AS price_matches_exactly,
  a.id, a.item_name, a.quantity, a.price, a.category_id, a.category_name
FROM expected e
LEFT JOIN actual a ON TRUE;

```

I checked this against four rows in SQLite (`sql/validate_logic.sh`): a correct one, one in Groceries, one with a `category_id` of 77 that matches no category, and a SKU that was never inserted. The wrong-category row comes back with `product_exists` true and `assigned_to_electronics` false, and the orphaned one with `category_reference_resolves` false, so the two are distinguishable. With an inner join both look the same as "never created".

A third query in the file lists products whose `category_id` matches no category at all. It should return nothing, and it runs nightly instead of per test.

Three changes to make once I know the real schema:

- If there is a tenant column, add it to the join as well as the where clause: `ON c.id = p.category_id AND c.tenant_id = p.tenant_id`. If categories are per-merchant and the join is on `category_id` alone it will attach another merchant's row, which a single-tenant test dataset never shows. It also allows the check that the same SKU returns nothing for any other tenant.
- If there are soft deletes, add `AND p.deleted_at IS NULL`, otherwise a row the API deleted still counts as found.
- If price is a float rather than `NUMERIC`, compare with a tolerance (`ABS(p.price - 25.00) < 0.005`), since 25.00 can be stored as 24.999999999. I would also raise storing money as a float separately.

In practice the database check runs after the API assertions and only looks at data the test created, over a read-only connection to a replica so the suite never holds locks on the primary. I would not have tests write to the database directly: that skips the business logic and turns into a second version of it.

2. CI/CD integration

Three workflows in `.github/workflows/`: `pr-checks.yml`, `post-deploy-e2e.yml` and `nightly.yml`. GitHub Actions here, but the same shape maps onto GitLab CI stages or a Jenkins pipeline.

```

commit / PR └─ typecheck (30s)
               └─ API tests (2m)
               └─ Postman/Newman (1m)
               └─ UI smoke (4m)          all green -> mergeable
                   |
                   merge to main, deploy to staging
                   |
                   UI smoke gate (3m) + DB integrity checks
                   |
                   full UI regression, 4 shards (~6m)
                   |
                   merged report, Slack message on failure

nightly: cross-browser, flake hunt (5x), dependency audit

```

Tests run in the Playwright container, or with `npx playwright install --with-deps`, so browser versions come from `package-lock.json` and not from whatever happens to be on the machine.

No credentials in the repository. URLs and the tenant come from environment variables; passwords and the read-only database URL come from GitHub Environment secrets, scoped per environment so a fork's PR cannot read staging credentials.

npm is cached by `setup-node`, and the browser binaries are cached under a key built from the resolved Playwright version, so an upgrade invalidates the cache instead of serving an old browser. It saves about 40 seconds a job.

The regression suite is split with `--shard=i/4` and the blob reports are combined with `playwright merge-reports`, so there is one HTML report rather than four. That takes it from around 24 minutes to about 6.

JUnit XML goes to the platform's test view, and the HTML report, screenshots, video and traces are uploaded on failure with retention set by usefulness: 3 days for shard blobs, 30 for the merged report. Because the trace is attached, I can usually work out a failure without reproducing it locally.

Branch protection requires typecheck, API tests, Newman and UI smoke before a merge. `forbidOnly` is on in CI, so a `test.only` left in a spec fails the build instead of quietly running one test and reporting green. Shards use `fail-fast: false` so one failing shard does not hide the others.

Test data is created and removed by each test through fixtures, with unique identifiers, so parallel jobs and repeated runs do not collide. There is no shared dataset to drift.

Failures post to the squad channel with a link to the run. The nightly repeat run is what keeps `retries: 1` honest, since it surfaces the flaky tests a retry would otherwise hide.

The suite lives in the same repository as the application and is reviewed the same way. A framework kept in a separate repo by one person tends to get worked around.

3. Execution strategy

The rule I follow is that the cheaper and more reliable a test is, the more often it should run.

Stage	Trigger	What runs	Time	Blocking
Static analysis	Every push	Typecheck, lint	< 1 min	Yes
API tests	Every push or PR	Full API suite and the Postman collection	2-3 min	Yes
UI smoke	Every PR	@smoke only	3-5 min	Yes
Post-deploy smoke	Deploy to staging	@smoke against the deployed app	3 min	Yes
UI regression	After the smoke gate	@regression, 4 shards	~6 min	Yes, for release
DB integrity	After deploy	Orphaned references, tenant isolation	1 min	Yes
Nightly	02:00 on weekdays	Cross-browser, repeat run, npm audit	30-45 min	No
Pre-release	Release branch	Everything, plus performance and security scans	60 min	Yes

All the API tests run on every commit because they take seconds, have no rendering to be fragile about, and cover the rules where the risk is on this platform: validation, permissions, tenant scoping, duplicate SKUs. There is no reason to sample them.

Only the smoke UI tests run per PR. UI tests are an order of magnitude slower and more fragile, and running 200 of them on every push adds little over the eight or so that would catch a genuinely broken build. It also teaches people to ignore CI.

The full UI regression runs after deployment, where there is a real network and real dependencies, and behind the smoke gate. If sign-in is broken there is no point spending 20 minutes finding that out in 200 other tests.

Cross-browser and the repeat run are nightly. WebKit matters here because merchants use iPads, but a WebKit-only regression is rare enough that finding it within a day is a reasonable trade for not paying for it on every push.

For a developer that means a broken validation rule comes back in about two minutes, a broken sign-in in about five, and a rendering regression in a rarely used screen the same day.

Notes and assumptions

1. No environment was provided, so the repository includes a small mock POS app (mock-app/, Node standard library only) implementing the endpoint from the brief along with sign-in, an inventory grid that loads asynchronously, roles, tenants and toasts that disappear after four seconds. It exists so the suite can be run, and so the waiting described in Part 1.4 has something real to wait for. Tests read `BASE_URL`, so the same suite points at staging.
2. Validation errors return 422 with `{ error: { code, message, details } }`. 400 would be equally reasonable; the tests check whichever is documented.
3. The database schema was not given, so the SQL uses only the two tables named in the brief and the columns from the payload. The three likely additions are listed in Part 3.1.
4. In the mock app SKUs are unique per tenant, not globally, which is what I would expect for a multi-tenant catalogue, and prices are held to two decimal places.
5. `category_id: 3` is Electronics, from the payload and the Part 3 question.